

INFORMAÇÕES		
MÓDULO	AULA	INSTRUTOR
09	Introdução ao React para desenvolvedores .NET	02 - Tipos, Funções, Organização de Estrutura e Importação de Recursos com TypeScript
		Pedro Henrique Amadio

CONTEÚDO
1. Relembrando rapidamente a Aula 01 Na aula passada, vimos que o TypeScript é uma extensão do JavaScript que adiciona tipagem estática, ajuda a evitar erros e torna o código mais previsível. Também criamos um projeto TypeScript puro, organizamos a pasta src, geramos o tsconfig.json e compilamos nosso primeiro exemplo. Nesta segunda aula, vamos avançar para um uso mais prático da linguagem, trabalhando com tipagem de funções, objetos, arrays, organização de pastas e importação de recursos.
2. Inferência de tipos Em muitos casos, o próprio TypeScript consegue deduzir o tipo automaticamente com base no valor atribuído. Isso é chamado de inferência de tipos. <pre>let nome = "Pedro"; let idade = 25; let ativo = true;</pre> Nesse exemplo, o TypeScript entende que nome é string, idade é number e ativo é boolean. A inferência reduz código repetitivo, mas a tipagem explícita continua importante quando queremos deixar contratos bem claros.
3. Tipando funções <pre>function somar(a: number, b: number): number { return a + b; } function exibirMensagem(nome: string): void { console.log(`Olá, \${nome}!`); } //Arrow function const multiplicar = (a: number, b: number): number => { return a * b; };</pre>

No TypeScript, podemos definir o tipo esperado para cada parâmetro e também o tipo de retorno. Isso evita chamadas incorretas e deixa a intenção do código muito mais legível.

4. Parâmetros opcionais

```
function saudacao(nome: string, sobrenome?: string): string {  
  if (sobrenome) {  
    return `Olá, ${nome} ${sobrenome}`;  
  }  
  
  return `Olá, ${nome}`;  
}
```

O ponto de interrogação indica que o parâmetro pode ou não ser informado. Esse recurso é muito comum em formulários, filtros e funções que recebem configurações.

5. Arrays e objetos tipados

```
let notas: number[] = [10, 8.5, 7, 9];  
let tecnologias: string[] = ["HTML", "CSS", "JavaScript", "TypeScript"];  
  
let aluno: { nome: string; idade: number } = {  
  nome: "Lucas",  
  idade: 22  
};
```

Arrays tipados garantem que todos os itens sigam o mesmo padrão. Já objetos tipados ajudam a controlar a estrutura dos dados, mas podem ficar repetitivos se a mesma definição for usada muitas vezes.

6. Reutilização com type e interface

Um **union type** é quando dizemos que uma variável, parâmetro ou propriedade pode aceitar **mais de um tipo possível**.

Para isso, usamos o operador `|`, que pode ser lido como “ou”.

Exemplo:

```
let codigo: string | number;
```

Isso significa:

- `codigo` pode ser uma `string`
- **ou** pode ser um `number`

Uso:

```
codigo = 123;  
codigo = "ABC-123";
```

Mas isto daria erro:

```
codigo = true;  
porque boolean não faz parte da união.
```

- Refinando o tipo com `typeof`

Podemos verificar qual tipo está armazenado naquele momento.

```
let valor: string | number;

valor = "Pedro";

if (typeof valor === "string") {
  console.log(valor.toUpperCase());
}
```

Agora funciona porque dentro do **if**, o **TypeScript** entende que **valor** é uma **string**.

Outro exemplo:

```
let resultado: string | number = 10;

if (typeof resultado === "number") {
  console.log(resultado * 2);
}
```

Esse processo de verificar o tipo antes de usar é muito importante em union types.

- **Union types com valores específicos**

Além de unir tipos como `string | number`, também podemos unir **valores literais**.

Exemplo:

```
let statusPedido: "pendente" | "aprovado" | "cancelado";
```

Agora essa variável só aceita exatamente esses três valores.

Uso:

```
statusPedido = "pendente";
statusPedido = "aprovado";
```

Mas isso daria erro:

```
statusPedido = "em analise";
```

Resumindo:

Com union types, uma variável pode aceitar mais de um tipo. Já as propriedades opcionais permitem estruturas flexíveis sem perder o controle da tipagem.

7. Boas práticas de organização

Até aqui, usamos um único arquivo para facilitar o aprendizado. Porém, em projetos reais, isso rapidamente vira bagunça. Uma boa prática é separar tipos e interfaces em uma pasta própria, deixando o arquivo principal mais limpo.

```
meu-projeto-ts/
├── src/
│   ├── index.ts
│   └── types/
│       ├── aluno.ts
│       ├── funcionario.ts
│       └── status.ts
├── dist/
├── package.json
└── tsconfig.json
```

Essa organização facilita leitura, manutenção, reaproveitamento e introduz o conceito de módulos, algo essencial em projetos modernos com JavaScript e TypeScript.

8. Criando e exportando tipos

Uma convenção simples para esta aula é usar uma pasta chamada types, guardando ali tanto type aliases quanto interfaces.

```
// src/types/aluno.ts
export interface Aluno {
  nome: string;
  idade: number;
  curso: string;
  matriculado: boolean;
}

// src/types/status.ts
export type Status = "ativo" | "inativo" | "bloqueado";
```

A palavra export torna o recurso disponível para outros arquivos. Sem ela, a interface ou o type ficaria restrito ao próprio arquivo.

9. Importando recursos no arquivo principal

```
import { Aluno } from "../types/aluno";
import { Status } from "../types/status";

const statusAtual: Status = "ativo";

const aluno1: Aluno = {
  nome: "Ana",
  idade: 21,
  curso: "TypeScript",
  matriculado: true
};

console.log(statusAtual);
console.log(aluno1);
```

O import traz um recurso de outro arquivo para uso no arquivo atual. Repare que não colocamos a extensão .ts no caminho do import.

10. Exemplo completo com organização

```
// src/types/aluno.ts
export interface Aluno {
  nome: string;
  idade: number;
  curso: string;
  matriculado: boolean;
}

// src/index.ts
import { Aluno } from "../types/aluno";
```

```

const alunos: Aluno[] = [];

for (let i = 0; i < 3; i++) {
  const nome = prompt("Nome do aluno:") || "";
  const idade = Number(prompt("Idade do aluno:"));
  const curso = prompt("Curso do aluno:") || "";
  const matriculado = prompt("Está matriculado? (s/n)") === "s";

  alunos.push({
    nome,
    idade,
    curso,
    matriculado
  });
}

console.log("Lista de alunos:");
console.log(alunos);

const alunosMatriculados = alunos.filter(aluno => aluno.matriculado);
console.log("Alunos matriculados:");
console.log(alunosMatriculados);

```

11. Atividade prática

Objetivo: criar um pequeno cadastro de alunos usando uma pasta types, uma interface separada em arquivo próprio, importação no index.ts e filtragem dos matriculados.

- Criar a estrutura src/index.ts e src/types/aluno.ts.
- Definir a interface Aluno no arquivo separado.
- Importar Aluno no index.ts.
- Cadastrar 3 alunos com prompt.
- Exibir a lista completa e filtrar somente os matriculados.

12. Exercícios de fixação

1. O que é um union type no TypeScript?

- Um tipo que aceita apenas números
- Um tipo que permite mais de uma possibilidade de tipo ou valor
- Um tipo usado apenas para arrays
- Um tipo exclusivo para funções

2. O que o símbolo | representa em TypeScript?

- Um comentário
- Uma condição
- Uma união entre tipos possíveis
- Um operador de importação

3. Analise o código abaixo:

```
let codigo: string | number;
```

O que isso significa?

- a) codigo só pode receber texto
- b) codigo só pode receber número
- c) codigo pode receber texto ou número
- d) codigo não pode receber valor algum

4. Qual das opções abaixo é um exemplo válido de union type?

a)

```
let idade: number = 20;
```

b)

```
let ativo: boolean = true;
```

c)

```
let id: string | number;
```

d)

```
let nome = "Pedro";
```

5. O que é uma propriedade opcional em uma interface?

- a) Uma propriedade que não pode ser usada
- b) Uma propriedade que pode ou não existir no objeto
- c) Uma propriedade que sempre será number
- d) Uma propriedade usada apenas em funções

6. Analise o código abaixo:

```
interface Usuario {  
  nome: string;  
  email: string;  
  telefone?: string;  
}
```

O que o `?` em `telefone?: string` indica?

- a) Que o campo é obrigatório
- b) Que o campo aceita apenas números
- c) Que o campo é opcional
- d) Que o campo foi removido da interface

7. Qual objeto abaixo está correto com base na interface a seguir?

```
interface Usuario {  
  nome: string;  
  email: string;  
  telefone?: string;  
}
```

a)

```
const usuario = {  
  nome: "Ana"  
};
```

b)

```
const usuario = {  
  nome: "Ana",  
  email: "ana@email.com"  
};
```

c)

```
const usuario = {  
  email: "ana@email.com",  
  telefone: "99999-9999"  
};
```

d)

```
const usuario = {  
  nome: "Ana",  
  email: 123  
};
```

8. Analise o código abaixo:

```
let statusPedido: "pendente" | "aprovado" | "cancelado";
```

Qual valor abaixo seria inválido?

- a) "pendente"
- b) "aprovado"
- c) "cancelado"
- d) "em análise"

9. Em qual situação faz sentido usar union type?

- a) Quando um valor pode assumir mais de um tipo ou mais de um estado específico
- b) Quando queremos apagar uma variável
- c) Quando queremos impedir o uso de interfaces
- d) Quando queremos transformar TypeScript em HTML

13. Desafio para casa

Criar a estrutura abaixo, tipando funcionários e status em arquivos separados, depois importar tudo no index.ts para exibir os dados e filtrar apenas os funcionários ativos.

```
src/  
├─ index.ts  
└─ types/
```

```
├─ funcionario.ts
└─ status.ts

// src/types/funcionario.ts
export interface Funcionario {
  nome: string;
  cargo: string;
  salario: number;
  ativo: boolean;
}

// src/types/status.ts
export type Status = "ativo" | "inativo";
```